Assignment_6_LAA_MDS202512

April 17, 2026

Linear Algebra and its Applications

Assignment 6

Submitted by: Arushi Marwaha

Course: MSc Data Science 1

Roll: MDS202512

Problem 1: Data Preparation

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,␣
 ↪classification_report

plt.style.use('seaborn-v0_8-muted')
plt.rcParams.update({
    'axes.spines.right': False,
    'axes.spines.top': False,
    'font.size': 10,
    'figure.titlesize': 14
})

#------------------------------ Data Preparation

# loading MNIST dataset
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')

# normalizing pixel values and converting labels
X, y = mnist.data / 255.0, mnist.target.astype(int)

# setting training size per class
```

```
N_TRAIN = 2855
X_test_size = 500

# splitting into train and test sets (stratified)
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=X_test_size, random_state=42, stratify=y
)

# constructing digit-wise matrices D(n)
digit_matrices = {}
for n in range(10):
    # selecting indices of digit n
    idx = np.where(y_train_full == n)[0][:N_TRAIN]
    # inserting images as columns
    digit_matrices[n] = X_train_full[idx].T

print(f"Matrices D(n) initialized. Training shape per class: {digit_matrices[0].
 ↪shape}")
```

```
Matrices D(n) initialized. Training shape per class: (784, 2855)
```

Problem 2: SVD and Principal Components

Question: What features do the leading singular vectors appear to capture?

The leading singular vectors, i.e., the first k columns of the matrix U in the decomposition

$$D(n) = U\Sigma V^\top,$$

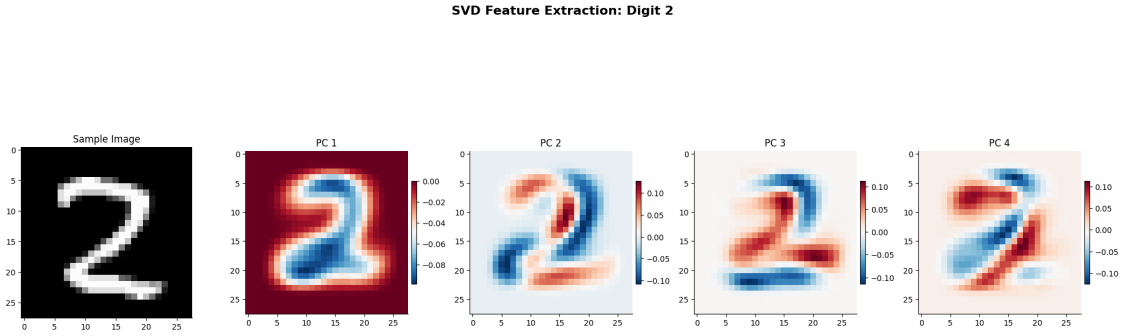
represent the **principal components** of the digit space.

These components capture the most significant patterns present in the data:

- **Structural Skeleton (PC1):**
  The first principal component typically represents the average or dominant structure of the digit. It captures the most common pixel arrangement across all samples in $D(n)$.
- **Variation in Form (PC2–PC4):**
  The subsequent components capture major variations in handwriting styles. For **Digit 2**, these include changes in slant, stroke thickness, and curvature.
- **Feature Offsets:**
  In the `RdBu_r` heatmaps, red and blue regions indicate positive and negative deviations from the base structure, highlighting how individual instances differ from the mean pattern.

[2]:
```
#------------------------------ SVD and Principal Components

# computing SVD for each class
svd_data = {}

for n in range(10):
    # performing SVD decomposition
```

```
    U, S, Vt = np.linalg.svd(digit_matrices[n], full_matrices=False)
    # storing U and singular values
    svd_data[n] = {'U': U, 'S': S}

# visualizing sample image and principal components
target_digit = 2
fig, axes = plt.subplots(1, 5, figsize=(20, 8))

fig.suptitle(f"SVD Feature Extraction: Digit {target_digit}",
             fontweight='bold', fontsize=16)

axes[0].imshow(digit_matrices[target_digit][:, 0].reshape(28, 28),
               cmap='gray', aspect='equal')
axes[0].set_title("Sample Image")

for i in range(1, 5):
    pc = svd_data[target_digit]['U'][:, i-1].reshape(28, 28)

    im = axes[i].imshow(pc, cmap='RdBu_r', aspect='equal')
    axes[i].set_title(f"PC {i}")

    plt.colorbar(im, ax=axes[i], fraction=0.03, pad=0.02)

plt.tight_layout()
plt.show()
```



Problem 3: Projection-based Classification

Question (a): Explain why this is equivalent to a least squares problem.

The classification rule is based on the residual

$$\rho_n(z) = \|z - U_k U_k^\top z\|_2.$$

This can be interpreted as a **linear least squares problem** as follows:

- We aim to approximate the test vector $z \in \mathbb{R}^{784}$ using a linear combination of the columns of U_k, i.e.,
$$z \approx U_k c,$$
where $c \in \mathbb{R}^k$.
- This leads to the least squares problem
$$\min_{c \in \mathbb{R}^k} \|z - U_k c\|_2.$$
- Since the columns of U_k are orthonormal, the optimal solution is given by
$$c^* = U_k^\top z.$$
- Substituting back, the best approximation (orthogonal projection) is
$$p = U_k c^* = U_k U_k^\top z.$$

Thus, the residual $\rho_n(z)$ represents the **minimum reconstruction error** of z from the subspace spanned by U_k, which is exactly the solution to a least squares problem.

Question (b): Why is this formulation computationally efficient?

- **Dimensionality Reduction:** Instead of comparing the test image z to every individual image in the training set, we only project z onto a very small k-dimensional subspace (e.g., $k = 32$).
- **Orthonormal Basis:** Because U_k is obtained from the SVD, its columns are orthonormal. This means we do not need to compute the pseudo-inverse $(U_k^T U_k)^{-1} U_k^T$; the projection is calculated via simple matrix-vector multiplications ($U_k^T z$ followed by U_k times the result), which is extremely fast.

[12]:
```
#------------------------------ Projection-Based Classification

# defining classifier using projection residual

def classify_svd(z, svd_dict, k):
    residuals = []
    for n in range(10):
        # taking first k basis vectors
        Uk = svd_dict[n]['U'][:, :k]
        # computing projection onto subspace
        projection = Uk @ (Uk.T @ z)
        # computing residual norm
        residual = np.linalg.norm(z - projection)
        residuals.append(residual)
    # returning class with minimum residual
    return np.argmin(residuals)
```

```
#------------------------------ Example Usage
k = 32
# classifying all test samples
preds = [classify_svd(img, svd_data, k) for img in X_test]

# computing accuracy
accuracy = np.mean(preds == y_test)

print(f"\nAccuracy (k={k}): {accuracy:.4f}")

# summary stats
print("\nSummary:")
print(f"Correct predictions: {np.sum(preds == y_test)} / {len(y_test)}")
```

```
Accuracy (k=32): 0.9520

Summary:
Correct predictions: 476 / 500
```

Problem 4: Effect of Rank k

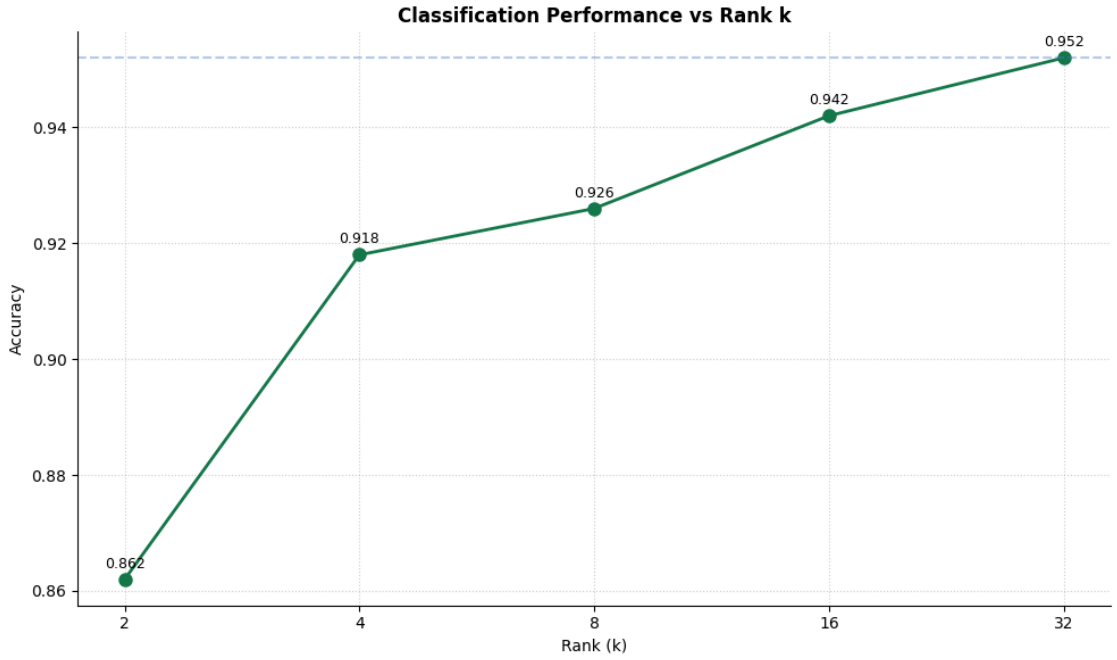
Question: Why does accuracy improve and then saturate?

Based on the generated results, the classification accuracy showed the following progression: * $k = 2$**:** 86.20% * $k = 32$**:** 95.20%

- **Improvement:** At low values of k, the subspace only captures the coarsest features of the digits. As k increases, the basis U_k incorporates more subtle variations (like the difference between a "straight" 7 and a "crossed" 7), allowing for a more precise fit and higher accuracy.
- **Saturation:** Accuracy saturates because the most significant "signal" of the digit is captured within the first few dozen singular values. Higher-order singular vectors correspond to very specific details or noise in the training set that do not necessarily help in generalizing to new test images.

[10]:
```
#------------------------------ Effect of Rank k

# evaluating accuracy for different k
ks = [2, 4, 8, 16, 32]
accuracies = []

for k in ks:
    # classifying all test samples
    preds = [classify_svd(img, svd_data, k) for img in X_test]

    # computing accuracy
    acc = np.mean(preds == y_test)
    accuracies.append(acc)
```

```python
#------------------------------ Plotting

plt.figure(figsize=(10, 6))
plt.plot(ks, accuracies, 'o-', color="#147749", linewidth=2, markersize=8)

# log scale for better spacing
plt.xscale('log', base=2)

for x, y in zip(ks, accuracies):
    plt.text(x, y + 0.002, f"{y:.3f}", ha='center', fontsize=9)

plt.axhline(y=max(accuracies), linestyle='--', alpha=0.4)
plt.title("Classification Performance vs Rank k", fontweight='bold')
plt.xlabel("Rank (k)")
plt.ylabel("Accuracy")
plt.xticks(ks, ks)
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
```



Problem 5: Energy Capture

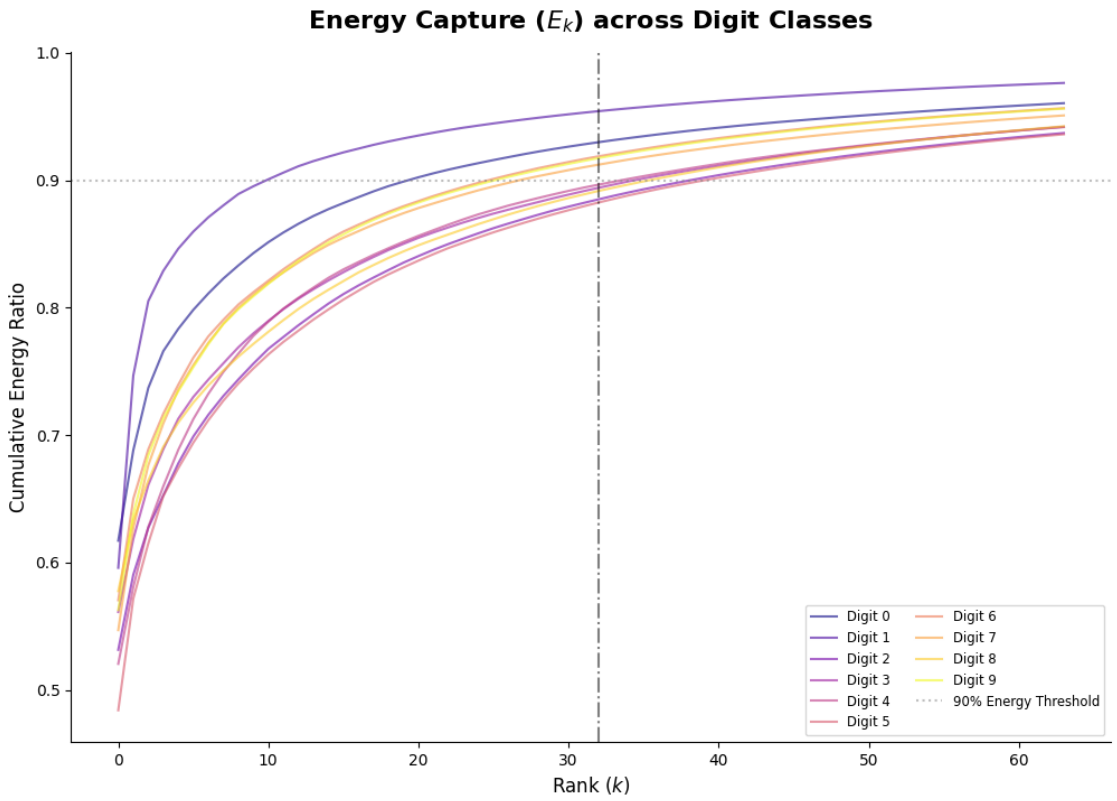
Question: How does energy relate to classification accuracy?

The energy capture E_k is defined as:

$$E_k = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_i \sigma_i^2}$$

- **Information Density:** High energy capture at low k indicates that the digit class is structurally consistent. For example, "Digit 1" typically reaches high E_k faster than "Digit 8" because it has less structural variety.
- **Correlation:** Accuracy generally tracks with the energy curve. When E_k reaches a threshold (usually around 80-90%), the subspace contains enough information to distinguish that digit from others reliably.

[5]:
```
#------------------------------ Energy Capture

# analyzing cumulative energy of singular values
plt.figure(figsize=(12, 8))
colors = plt.cm.plasma(np.linspace(0, 1, 10))

for n in range(10):
    s = svd_data[n]['S']
    # computing cumulative energy ratio
    energy = np.cumsum(s**2) / np.sum(s**2)
    plt.plot(energy[:64], color=colors[n], alpha=0.6, label=f"Digit {n}")

# inserting reference line at k = 32
plt.axvline(x=32, color='black', linestyle='-.', alpha=0.5)
plt.axhline(y=0.9, color='gray', linestyle=':', alpha=0.5, label='90% Energy␣
 ↪Threshold')

plt.title("Energy Capture ($E_k$) across Digit Classes", fontsize=16,␣
 ↪fontweight='bold', pad=15)
plt.xlabel("Rank ($k$)", fontsize=12)
plt.ylabel("Cumulative Energy Ratio", fontsize=12)
plt.legend(ncol=2, fontsize='small')
plt.show()
```



Problem 6: Noise Robustness

Question: Why might smaller k perform better in noisy settings?

From the results:

- $k = 4$
  Clean Accuracy: 0.9180, Noisy Accuracy: 0.8800 (Drop: 0.0380)
- $k = 32$
  Clean Accuracy: 0.9520, Noisy Accuracy: 0.8720 (Drop: 0.0800)

Although larger k gives higher accuracy on clean data, it shows a greater performance drop under noise, whereas smaller k remains more stable.

This can be explained using the SVD structure. Let

$$D^{(n)} = U\Sigma V^\top,$$

where the columns of U form an orthonormal basis. The leading singular vectors (associated with large singular values σ_i) capture the dominant signal, while higher-order singular vectors correspond to low-energy components that are more sensitive to noise.

If a noisy image is given by

$$z = z_{\text{true}} + \eta,$$

where $\eta \sim \mathcal{N}(0, \sigma^2 I)$ is Gaussian noise, then the projection onto the rank-k subspace is

$$\hat{z} = U_k U_k^\top z = U_k U_k^\top z_{\text{true}} + U_k U_k^\top \eta.$$

For smaller k, the subspace spanned by U_k primarily contains directions corresponding to large singular values, so

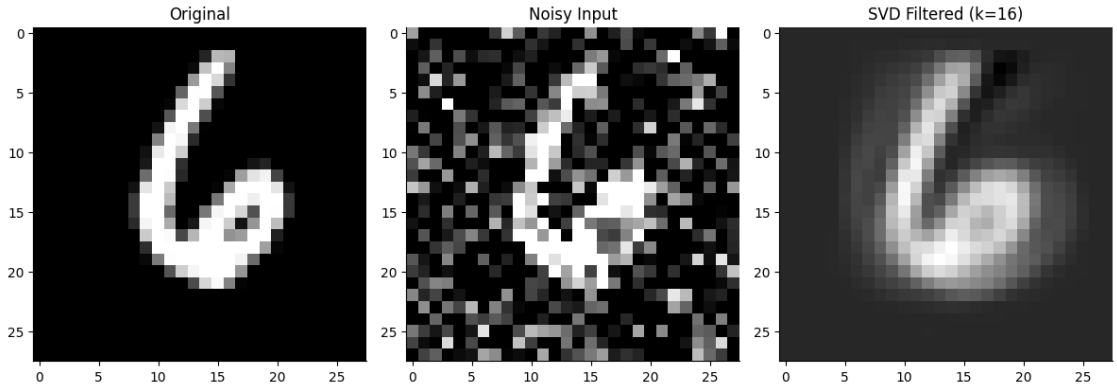
$$\|U_k U_k^\top \eta\|$$

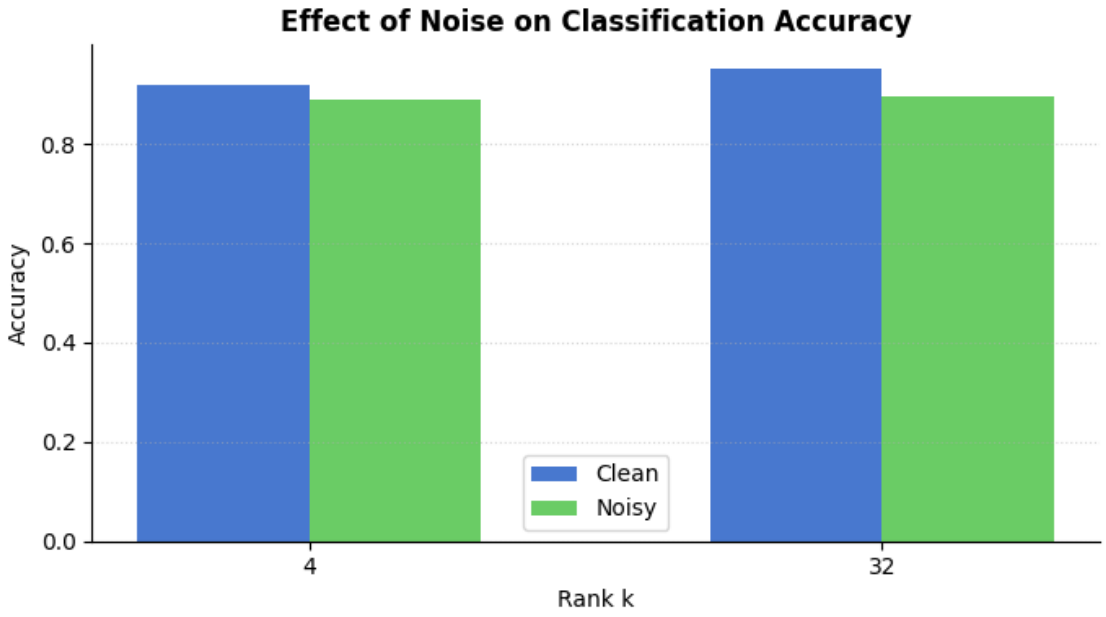
is small, and most of the noise is filtered out. In contrast, larger k includes additional directions associated with smaller singular values, where noise has a relatively larger effect.

Thus, smaller k effectively acts as a low-rank approximation that suppresses noise, leading to improved robustness, as confirmed by the smaller accuracy drop observed in the results.

[11]:
```
#------------------------------ Noise Robustness

# adding Gaussian noise to test data

noise_factor = 0.4
X_test_noisy = np.clip(
    X_test + noise_factor * np.random.normal(size=X_test.shape),
    0, 1
)

#------------------------------ Visual Comparison

# selecting a sample image
sample_idx = 310

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 4))

# original image
ax1.imshow(X_test[sample_idx].reshape(28,28), cmap='gray')
ax1.set_title("Original")

# noisy image
ax2.imshow(X_test_noisy[sample_idx].reshape(28,28), cmap='gray')
ax2.set_title("Noisy Input")

# reconstruction using k = 16 (denoising effect)
U_k = svd_data[y_test[sample_idx]]['U'][:, :16]
filtered = U_k @ (U_k.T @ X_test_noisy[sample_idx])

ax3.imshow(filtered.reshape(28,28), cmap='gray')
```

```
ax3.set_title("SVD Filtered (k=16)")

plt.tight_layout()
plt.show()


#------------------------------ Accuracy Comparison


ks_compare = [4, 32]

clean_accs = []
noisy_accs = []

for k_val in ks_compare:
    # clean predictions
    clean_preds = [classify_svd(img, svd_data, k_val) for img in X_test]
    clean_acc = np.mean(clean_preds == y_test)
    clean_accs.append(clean_acc)

    # noisy predictions (reclassification)
    noisy_preds = [classify_svd(img, svd_data, k_val) for img in X_test_noisy]
    noisy_acc = np.mean(noisy_preds == y_test)
    noisy_accs.append(noisy_acc)



#------------------------------ Accuracy Visualization

x = np.arange(len(ks_compare))

plt.figure(figsize=(7, 4))

plt.bar(x - 0.15, clean_accs, width=0.3, label='Clean')
plt.bar(x + 0.15, noisy_accs, width=0.3, label='Noisy')

plt.xticks(x, ks_compare)
plt.xlabel("Rank k")
plt.ylabel("Accuracy")

plt.title("Effect of Noise on Classification Accuracy", fontweight='bold')

plt.grid(axis='y', linestyle=':', alpha=0.5)
plt.legend()

plt.tight_layout()
plt.show()
```





Problem 7: Results Summary

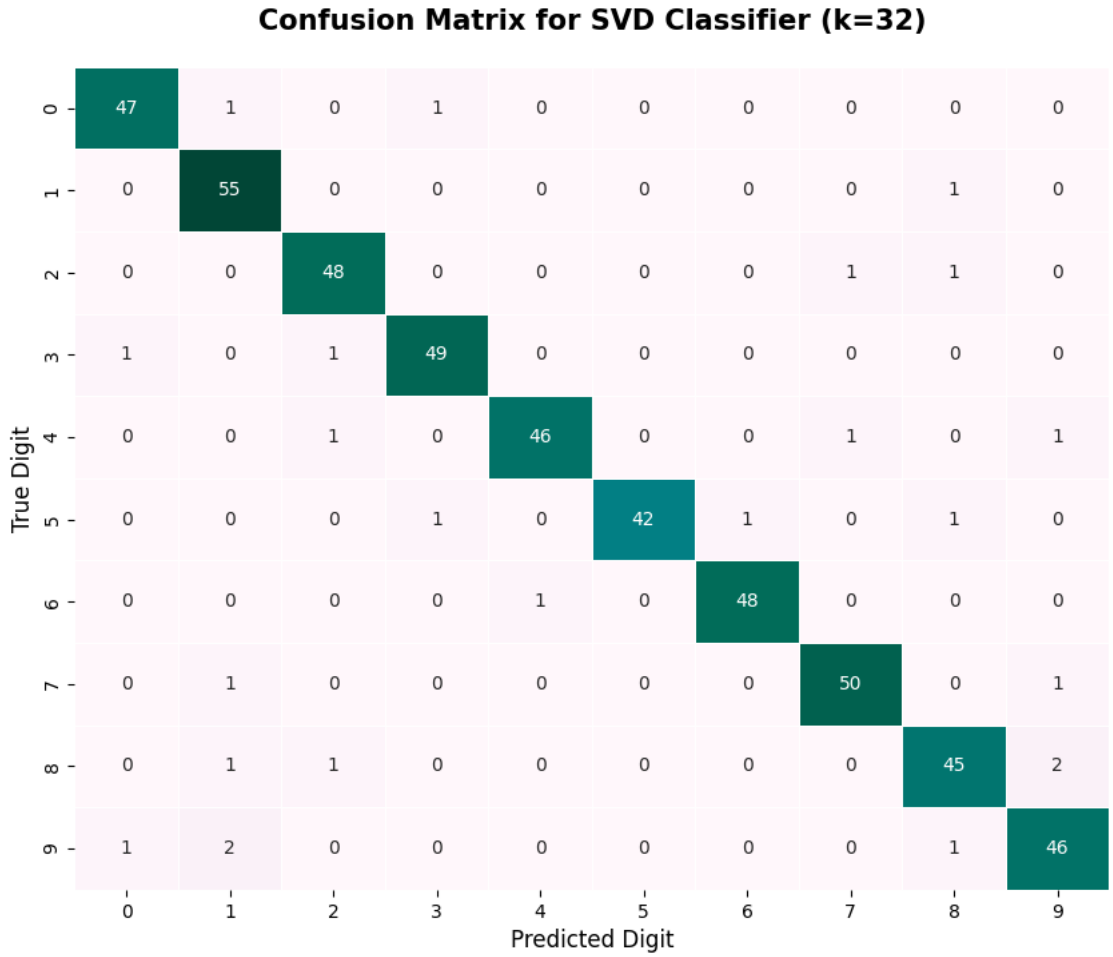
The final evaluation at $k = 32$ yielded a global accuracy of **95%**.

- **Confusion Matrix:** Most digits were correctly identified, with some minor confusion between structurally similar digits (e.g., 4 and 9).
- **Reconstruction:** The progression of images from $k = 2$ to $k = 32$ shows the visual "assembly" of the digit as more algebraic information is added through the principal components.

[7]:
```
#------------------------------ RESULTS
final_k = 32
final_preds = [classify_svd(img, svd_data, final_k) for img in X_test]
```

```
# Confusion Matrix Visualization
cm = confusion_matrix(y_test, final_preds)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='PuBuGn', cbar=False, linewidths=.5)

plt.title(f"Confusion Matrix for SVD Classifier (k={final_k})", fontsize=15,␣
 ↪fontweight='bold', pad=20)
plt.xlabel("Predicted Digit", fontsize=12)
plt.ylabel("True Digit", fontsize=12)
plt.show()

#------------------------------ Classification Report

print("\nClassification Report:")

report = classification_report(y_test, final_preds, output_dict=True)
# extracting metrics
print("-" * 70)
print(f"{'Digit':<10}{'Precision':<12}{'Recall':<12}{'F1-score':<12}{'Support':
 ↪<10}")
print("-" * 70)

for digit in range(10):
    d = report[str(digit)]
    print(f"{digit:<10}{d['precision']:<12.2f}{d['recall']:<12.
 ↪2f}{d['f1-score']:<12.2f}{int(d['support']):<10}")

print("-" * 70)

# overall metrics
print(f"{'Accuracy':<10}{report['accuracy']:<12.2f}")
print(f"{'Macro Avg':<10}{report['macro avg']['precision']:<12.
 ↪2f}{report['macro avg']['recall']:<12.2f}{report['macro avg']['f1-score']:
 ↪<12.2f}")
print(f"{'Weighted':<10}{report['weighted avg']['precision']:<12.
 ↪2f}{report['weighted avg']['recall']:<12.2f}{report['weighted␣
 ↪avg']['f1-score']:<12.2f}")
print("-" * 70)

# Selecting a test image for visualization
three_indices = np.where(y_test == 3)[0]
test_img_3 = X_test[three_indices[0]]

fig, axes = plt.subplots(1, len(ks), figsize=(18, 5))
fig.suptitle(f"Image Reconstruction Progression for Digit 3", fontsize=16,␣
 ↪fontweight='bold', y=1.05)
```

```python
for i, k in enumerate(ks):
    # Orthogonal projection onto the k-dimensional subspace: Uk * Uk^T * z
    U_k = svd_data[3]['U'][:, :k]
    recon = U_k @ (U_k.T @ test_img_3)

    axes[i].imshow(recon.reshape(28, 28), cmap='gray')
    axes[i].set_title(f"Rank k = {k}", fontsize=12, fontweight='semibold')
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```




```
Classification Report:
----------------------------------------------------------------------
Digit      Precision    Recall       F1-score     Support
```

0	0.96	0.96	0.96	49
1	0.92	0.98	0.95	56
2	0.94	0.96	0.95	50
3	0.96	0.96	0.96	51
4	0.98	0.94	0.96	49
5	1.00	0.93	0.97	45
6	0.98	0.98	0.98	49
7	0.96	0.96	0.96	52
8	0.92	0.92	0.92	49
9	0.92	0.92	0.92	50
Accuracy	0.95			
Macro Avg	0.95	0.95	0.95	
Weighted	0.95	0.95	0.95